

DOCUMENT 01 – PRODUCT REQUIREMENTS DOCUMENT (PRD)

Project Name

Postman Collection → Pytest Migrator

Tagline

Transform Postman API collections into executable pytest automation suites using Gemini AI.

Problem Statement

Many software teams use Postman to create and validate API requests during development and testing.

Although Postman supports API testing, the test logic remains inside Postman collections and cannot be directly used in Python-based automation frameworks such as pytest.

QA engineers and developers often spend significant time manually converting Postman requests and assertions into pytest automation scripts.

This manual process:

- Consumes development time
- Introduces human errors
- Creates maintenance challenges
- Delays automation adoption

The proposed solution automates this conversion process using Gemini AI.

The system reads a Postman Collection JSON file and automatically generates pytest functions with migrated assertions.

Project Objective

Develop an AI-assisted platform that:

1. Accepts Postman Collection JSON files
2. Extracts API requests and assertions

3. Uses Gemini AI to understand test logic
 4. Generates executable pytest functions
 5. Produces downloadable automation scripts
-

Target Users

QA Automation Engineers

Need faster migration from Postman collections to pytest automation.

Pain Points

- Manual test script creation
 - Repetitive conversion work
 - Increased maintenance effort
-

Backend Developers

Need automated API validation during development.

Pain Points

- Lack of reusable automation scripts
 - Manual verification of API responses
-

DevOps Engineers

Need automated test execution in CI/CD pipelines.

Pain Points

- Difficulty integrating Postman tests into deployment workflows
-

Core Value Proposition

The application uses Gemini AI to understand:

- API Requests
- HTTP Methods

- Headers
- Request Bodies
- Authentication
- Variables
- Postman Assertions

and automatically generates clean, maintainable pytest automation code.

Must-Have Features

Collection Upload

Upload Postman Collection JSON files.

Functions

- File upload
 - JSON validation
 - Collection verification
-

Collection Parser

Extract:

- Requests
 - Methods
 - URLs
 - Headers
 - Variables
 - Authentication
 - Postman Assertions
-

AI Conversion Engine

Gemini AI converts Postman assertions into pytest assertions.

Example:

Postman:

```
pm.response.to.have.status(200);
```

Converted Output:

```
assert response.status_code == 200
```

Validation Engine

Verify:

- Assertion syntax
 - Unsupported Postman functions
 - Missing variables
 - Invalid structures
-

Pytest Generator

Generate:

- Test Functions
 - Assertions
 - Request Templates
 - Executable Pytest Scripts
-

Download Module

Export:

- Python Files (.py)
 - ZIP Packages
-

Conversion Report

Display:

- Converted Assertions
 - Failed Conversions
 - Warnings
 - Conversion Statistics
-

User Stories

Story 1

As a QA Engineer,

I want to upload a Postman Collection,

So that pytest automation scripts are generated automatically.

Story 2

As a Developer,

I want generated pytest scripts,

So that I can integrate them into CI/CD pipelines.

Story 3

As a Test Architect,

I want reusable pytest code,

So that automation standards are maintained across projects.

AI Capability Demonstration

Capability

Specification → Code Translation

Input

- Postman Collection JSON
 - Postman Assertions
-

Output

- Executable Pytest Functions
-

Agent Loop

Parser Agent

↓

Analysis Agent

↓

Gemini Conversion Agent

↓

Validation Agent

↓

Export Agent

Technology Stack

Frontend

- HTML5
 - CSS3
 - JavaScript
-

Backend

- Python 3.11
 - Flask
-

Database

- SQLite
-

AI

- Gemini API (Free Tier)
-

Source Control

- GitHub
-

Success Metrics

- 95%+ Assertion Conversion Accuracy
 - 90%+ Successful Script Generation
 - 80% Reduction in Manual Conversion Effort
 - Processing Time Less Than 30 Seconds per Collection
-

Expected Outcome

The system should successfully convert Postman Collection requests and assertions into executable pytest automation scripts using Gemini AI, reducing manual effort and improving QA productivity.

Technical Requirements Document (TRD)

Project Title

Postman Collection to Pytest Migrator

Version

1.0

Project Overview

Postman Collection to Pytest Migrator is an AI-assisted QA automation platform designed to reduce manual effort in API testing. The system accepts a Postman Collection JSON file as input and automatically generates API test cases, pytest automation scripts, and AI-powered testing recommendations.

The application is intended for QA engineers, software testers, and developers who need to create automation tests quickly without manually writing repetitive test scripts.

Technology Stack

Frontend Technologies

HTML5

Used for building the user interface and file upload forms.

CSS3

Used for application styling and responsive design.

JavaScript

Used for frontend validation, file handling, and user interactions.

Backend Technologies

Python 3.11

Primary programming language used for:

- File processing
- JSON parsing
- Test case generation
- Pytest script generation
- AI integration

Flask Framework

Used for:

- API development
- Request handling
- File upload management
- Response generation

Reason for Selection:

- Lightweight
 - Easy to learn
 - Suitable for rapid development
 - Open source
-

AI Technology

Google Gemini API (Free Tier)

Purpose:

- Analyze uploaded API definitions
- Generate intelligent test scenarios
- Generate positive and negative test cases
- Suggest edge-case testing

Examples:

Input: POST /login

Output:

- Valid Login Test
 - Invalid Password Test
 - Empty Email Test
 - SQL Injection Test
 - Special Character Validation Test
-

Source Control

GitHub

Purpose:

- Version control
- Team collaboration
- Project hosting
- Submission repository

Repository Type: Public Repository

Data Source

Postman Collection JSON

Input format accepted:

.json

Example Contents:

- API Name
 - Request Method
 - Endpoint URL
 - Request Body
 - Headers
 - Parameters
-

Application Modules

Module 1: Collection Upload

Functions:

- Upload JSON collection
 - Validate file format
 - Validate file size
 - Store temporary file
-

Module 2: Collection Parser

Functions:

- Read JSON file
- Extract API details
- Extract endpoints
- Extract request bodies
- Extract headers

Output:

Structured API metadata

Module 3: Test Case Generator

Functions:

- Generate positive test cases
 - Generate negative test cases
 - Generate validation test cases
 - Generate boundary test cases
-

Module 4: AI Recommendation Engine

Functions:

- Analyze API structure
- Generate additional QA scenarios
- Suggest missing validations

- Identify security test opportunities
-

Module 5: Pytest Script Generator

Functions:

- Create pytest functions
 - Create API request templates
 - Generate assertions
 - Generate executable test scripts
-

Module 6: Download Center

Functions:

- Download generated test cases
 - Download pytest scripts
 - Download AI recommendations
-

System Workflow

Step 1: User uploads Postman Collection

↓

Step 2: System validates JSON file

↓

Step 3: Collection Parser extracts API information

↓

Step 4: AI analyzes API definitions

↓

Step 5: Test cases generated

↓

Step 6: Pytest scripts generated



Step 7: Results displayed



Step 8: User downloads output files

External Integrations

Google Gemini API

Purpose: AI-powered test generation

Type: External Public API

Cost: Free Tier

File Structure

project/

├── app.py

├── requirements.txt

├── uploads/

├── generated_tests/

├── generated_scripts/

├── templates/

├── static/

├── parser/

└── ai_engine/

|—— utils/

|—— README.md

Environment Variables

GEMINI_API_KEY

Security Considerations

- Validate uploaded files
 - Accept JSON files only
 - Protect API keys using environment variables
 - Prevent malicious file uploads
-

Constraints

- Development duration limited to 1–2 days
 - Only free/open-source tools
 - Must demonstrate AI capability
 - Must maintain prompt documentation
 - Must be deployable locally
 - Must support Postman Collection v2.1
-

Expected Deliverables

1. Working Application
 2. Public GitHub Repository
 3. README Documentation
 4. AI Usage Note
 5. Sample Data
 6. Test Cases
 7. Demo Video
-

Expected Outcome

The system will reduce manual QA effort by automatically generating API test cases and pytest automation scripts from Postman collections while demonstrating AI-assisted software development.

DOCUMENT 03 – APPLICATION FLOW

Project Name

Postman Collection → Pytest Migrator

Navigation Type

Single Page Dashboard Application

Navigation Components:

- Top Navigation Bar
 - Main Content Area
 - Results Section
 - Download Section
-

User Roles

User

Permissions:

- Upload Postman Collection
 - Generate Pytest Scripts
 - View Conversion Results
 - Download Generated Files
-

Admin

Permissions:

- View Uploaded Collections
 - Monitor Conversion History
 - View System Statistics
-

Application Screens

Screen 1: Landing Page

Purpose

Introduce the project and allow users to begin conversion.

Components

- Project Title
- Project Description
- Upload Button
- Generate Button

User Actions

- Select Collection File
 - Upload Collection
-

Screen 2: Collection Upload

Purpose

Upload and validate Postman Collection JSON.

Components

- File Selector
- Upload Status
- Validation Messages

Validation Rules

- Accept JSON files only
- Reject empty files
- Verify Postman Collection structure

Output

Validated Collection

Screen 3: Collection Analysis

Purpose

Display extracted collection information.

Information Displayed

- Collection Name
- Total Requests
- HTTP Methods
- Endpoints
- Variables
- Assertions Found

User Actions

- Review extracted information
 - Continue to conversion
-

Screen 4: AI Conversion Dashboard

Purpose

Convert Postman assertions into pytest code using Gemini AI.

Process Flow

Collection Data

↓

Parser Engine

↓

Gemini AI

↓

Pytest Generator

↓

Validation Engine

Display

- Processing Status
 - Conversion Progress
 - Converted Assertions Count
-

Screen 5: Generated Code Viewer

Purpose

Display generated pytest code.

Components

- Syntax Highlighted Code
- Test Function Preview
- Assertion Preview

Example

Postman Assertion:

```
pm.response.to.have.status(200)
```

Generated Pytest:

```
assert response.status_code == 200
```

Screen 6: Conversion Report

Purpose

Display conversion statistics.

Information

- Total APIs Processed
 - Assertions Converted
 - Failed Conversions
 - Warnings
 - Success Percentage
-

Screen 7: Download Center

Purpose

Download generated output files.

Available Downloads

- Pytest Script (.py)
- ZIP Package
- Conversion Report

User Actions

- Download Files
 - Generate Again
-

Core User Journey

Step 1

User uploads Postman Collection JSON

↓

Step 2

System validates file

↓

Step 3

Parser extracts requests and assertions

↓

Step 4

Gemini AI converts Postman assertions

↓

Step 5

Pytest Generator creates automation scripts



Step 6

Validation Engine checks generated code



Step 7

Results displayed



Step 8

User downloads generated files

Error Handling Flow

Invalid JSON

Display:

"Invalid Postman Collection Format"

Parsing Failure

Display:

"Unable to Parse Collection"

Gemini API Failure

Display:

"AI Conversion Service Temporarily Unavailable"

Generation Failure

Display:

"Unable to Generate Pytest Script"

Success Flow

Collection Uploaded

↓

Collection Parsed

↓

Assertions Extracted

↓

AI Conversion Successful

↓

Pytest Script Generated

↓

Download Available

End-to-End Workflow

Postman Collection

↓

Upload Module

↓

Collection Parser

↓

Assertion Extractor

↓

Gemini AI Conversion

↓

Pytest Generator

↓

Validation Engine

↓

SQLite Storage

↓

Results Page

↓

Download Center

Expected User Outcome

The user should be able to upload a Postman Collection and receive an executable pytest automation script with converted assertions in a simple and intuitive workflow.

DOCUMENT 04 – UI/UX DESIGN BRIEF

Project Name

Postman Collection → Pytest Migrator

Design Philosophy

The application should provide a simple and intuitive experience for QA engineers and developers.

The primary goal is to allow users to upload a Postman Collection and obtain pytest automation scripts with minimal effort.

The interface should be:

- Clean
 - Fast
 - Professional
 - Easy to understand
 - Focused on productivity
-

Design Style

Theme

Modern Developer Dashboard

Inspired By:

- Postman
- GitHub
- Swagger UI
- Vercel

Characteristics:

- Minimal Design
- Developer Friendly
- Fast Navigation
- Simple Workflow

- Clear Visual Hierarchy
-

Color Palette

Primary Color

Indigo

Hex Code:

4F46E5

Usage:

- Primary Buttons
 - Navigation Elements
 - Active Sections
-

Secondary Color

Teal

Hex Code:

14B8A6

Usage:

- Success Messages
 - Completed Conversions
 - Download Status
-

Background Color

Dark Slate

Hex Code:

0F172A

Usage:

- Main Background
 - Dashboard Area
-

Text Color

Light Gray

Hex Code:

F8FAFC

Usage:

- Headings
 - Content
 - Navigation Text
-

Error Color

Red

Hex Code:

EF4444

Usage:

- Validation Errors
 - Conversion Failures
 - API Errors
-

Typography

Primary Font

Inter

Reasons:

- Modern appearance
 - Excellent readability
 - Developer-friendly design
-

Font Sizes

Main Heading

32px

Section Heading

24px

Body Text

16px

Small Text

14px

Layout Structure

Header

Contains:

- Project Name
- Navigation Links

- Theme Toggle (Optional)
-

Main Content Area

Contains:

- Upload Section
 - Analysis Section
 - Conversion Section
 - Results Section
-

Footer

Contains:

- Version Information
 - Team Information
 - Copyright
-

Main Screens

Screen 1: Landing Page

Purpose:

Introduce the application.

Components:

- Project Logo
 - Project Description
 - Upload Button
 - Quick Start Guide
-

Screen 2: Upload Collection

Purpose:

Upload Postman Collection JSON.

Components:

- Drag and Drop Area
- File Selector
- Upload Status

Actions:

- Upload File
 - Validate File
-

Screen 3: Collection Analysis

Purpose:

Display extracted collection information.

Components:

- API Count
- Request Methods
- Endpoints
- Assertions

Display Format:

Card-Based Layout

Screen 4: AI Conversion Dashboard

Purpose:

Show Gemini AI conversion progress.

Components:

- Progress Bar
- Conversion Status
- Converted Assertions Count

Display Messages:

- Parsing Collection
- Calling Gemini AI

- Generating Pytest Code
 - Validating Output
-

Screen 5: Generated Code Viewer

Purpose:

Display generated pytest code.

Components:

- Code Editor View
- Syntax Highlighting
- Copy Button

Features:

- Read-only Mode
 - Scroll Support
 - Search Support
-

Screen 6: Conversion Report

Purpose:

Display conversion statistics.

Information:

- APIs Processed
- Assertions Converted
- Failed Conversions
- Success Rate

Display Type:

Summary Cards

Screen 7: Download Center

Purpose:

Download generated files.

Components:

- Download Python File
 - Download ZIP Package
 - Regenerate Button
-

User Experience Guidelines

Simplicity

Users should complete the conversion process in less than three clicks.

Workflow:

Upload

↓

Generate

↓

Download

Visibility

Display system progress at every stage.

Examples:

- Uploading Collection
 - Parsing Collection
 - Converting Assertions
 - Generating Scripts
-

Feedback

Every action should provide visual feedback.

Examples:

- Success Messages
 - Error Notifications
 - Loading Indicators
-

Responsive Design

Desktop

Full Dashboard Layout

Minimum Width:

1200px

Tablet

Adaptive Layout

Minimum Width:

768px

Mobile

Single Column Layout

Minimum Width:

320px

Accessibility

Requirements:

- Keyboard Navigation
- High Contrast Support
- Readable Typography

- Proper Form Labels
 - Screen Reader Compatibility
-

Performance Targets

First Page Load:

Less than 2 Seconds

File Upload Validation:

Less than 1 Second

Pytest Generation:

Less than 30 Seconds

UI Response Time:

Less than 100 ms

Expected User Experience

Users should be able to upload a Postman Collection, view extracted API information, generate pytest automation scripts using Gemini AI, and download the generated files through a clean, responsive, and professional interface.

Document 05 – Backend Schema & Architecture

Project Name

Postman Collection to Pytest Migrator

1. Backend Overview

The backend is responsible for processing uploaded Postman Collection JSON files, extracting API information, generating test cases, creating pytest automation scripts, storing generated results, and providing AI-powered testing recommendations.

The system follows a modular architecture to ensure scalability, maintainability, and efficient data management.

2. Backend Architecture

User ↓ Frontend (HTML, CSS, JavaScript) ↓ Flask Backend ↓ Collection Parser ↓ Gemini AI Engine ↓ SQLite Database ↓ Generated Results

3. Database Technology

Database: SQLite

Reason for Selection:

- Open Source
 - Lightweight
 - Easy Integration with Python
 - Suitable for Hackathon Development
 - No Separate Database Server Required
-

4. Database Tables

Table 1: uploaded_collections

Purpose: Stores information about uploaded Postman Collection files.

Field Name	Data Type	Description
id	INTEGER	Primary Key
collection_name	TEXT	Name of uploaded collection
file_name	TEXT	Original file name
uploaded_by	TEXT	User name
uploaded_at	DATETIME	Upload timestamp
total_apis	INTEGER	Number of APIs detected
status	TEXT	Processing status

Table 2: api_details

Purpose: Stores API information extracted from Postman collections.

Field Name	Data Type	Description
id	INTEGER	Primary Key
collection_id	INTEGER	Foreign Key
api_name	TEXT	Name of API
method	TEXT	GET, POST, PUT, DELETE
endpoint	TEXT	API URL
headers	TEXT	Request headers
request_body	TEXT	Request payload
query_params	TEXT	Query parameters

Table 3: generated_testcases

Purpose: Stores generated QA test cases.

Field Name	Data Type	Description
id	INTEGER	Primary Key
api_id	INTEGER	Foreign Key
testcase_name	TEXT	Name of test case

Field Name	Data Type	Description
testcase_type	TEXT	Positive, Negative, Boundary
expected_result	TEXT	Expected output
generated_at	DATETIME	Creation timestamp

Table 4: generated_scripts

Purpose: Stores generated pytest automation scripts.

Field Name	Data Type	Description
id	INTEGER	Primary Key
api_id	INTEGER	Foreign Key
script_name	TEXT	Generated script name
script_content	TEXT	Pytest code
created_at	DATETIME	Generation timestamp

Table 5: ai_recommendations

Purpose: Stores AI-generated testing recommendations.

Field Name	Data Type	Description
id	INTEGER	Primary Key
api_id	INTEGER	Foreign Key
recommendation	TEXT	AI suggestion
recommendation_type	TEXT	Functional, Security, Performance
generated_at	DATETIME	Creation timestamp

5. Database Relationships

uploaded_collections ↓ api_details ↓ | generated_testcases generated_scripts ai_recommendations ↓ ↓ ↓

Relationship Type:

- One Collection → Many APIs
 - One API → Many Test Cases
 - One API → Many Scripts
 - One API → Many AI Recommendations
-

6. Backend Modules

Module 1: Collection Upload Module

Responsibilities:

- Upload JSON files
 - Validate file format
 - Store collection details
 - Save collection metadata
-

Module 2: Collection Parser Module

Responsibilities:

- Parse Postman Collection
 - Extract API details
 - Extract request information
 - Store API metadata
-

Module 3: AI Recommendation Engine

Technology: Google Gemini API

Responsibilities:

- Analyze APIs
 - Generate testing suggestions
 - Generate security checks
 - Suggest edge cases
-

Module 4: Test Case Generator

Responsibilities:

- Generate Positive Test Cases
 - Generate Negative Test Cases
 - Generate Boundary Test Cases
 - Store generated results
-

Module 5: Pytest Script Generator

Responsibilities:

- Generate executable pytest scripts
 - Create assertions
 - Generate reusable automation code
 - Save generated scripts
-

Module 6: Result Management Module

Responsibilities:

- Retrieve stored results
 - Display generated outputs
 - Download scripts
 - Download test cases
-

7. API Endpoints

Upload Collection

POST /upload

Purpose: Upload Postman Collection JSON file.

Extract APIs

POST /extract

Purpose: Extract API information from uploaded collection.

Generate Test Cases

POST /generate-testcases

Purpose: Generate QA test cases.

Generate Pytest Scripts

POST /generate-pytest

Purpose: Generate automation scripts.

Generate AI Suggestions

POST /generate-recommendations

Purpose: Generate AI-powered testing recommendations.

View Results

GET /results

Purpose: Display generated outputs.

Download Output

GET /download

Purpose: Download generated files.

8. Security Considerations

- JSON File Validation
- File Size Validation
- Secure API Key Storage

- Environment Variable Configuration
 - Input Sanitization
 - Error Handling Mechanism
-

9. User Roles

User

Permissions:

- Upload Collection
- Generate Test Cases
- Generate Scripts
- View Results
- Download Outputs

Admin

Permissions:

- Manage Collections
 - View All Results
 - Monitor System Activity
-

10. Expected Backend Outcome

The backend system should successfully process Postman Collection files, extract API information, generate QA test cases, create pytest automation scripts, store generated results in SQLite, and provide AI-powered testing recommendations through an efficient and scalable architecture.

DOCUMENT 06 – IMPLEMENTATION PLAN

Project Name

Postman Collection → Pytest Migrator

Project Goal

Develop an AI-assisted application that converts Postman Collection JSON files into executable pytest automation scripts using Gemini AI.

The system should:

- Upload Postman Collections
 - Extract API requests and assertions
 - Convert Postman assertions into pytest assertions
 - Generate executable pytest functions
 - Store results in SQLite
 - Allow download of generated files
-

Development Timeline

Duration

1–2 Days

Phase 1 – Project Setup

Objective

Prepare development environment and project structure.

Tasks

- Create GitHub Repository
- Create Flask Project
- Configure Virtual Environment
- Install Required Packages

- Configure Gemini API Key
- Configure SQLite Database

Deliverables

- Working Flask Application
 - GitHub Repository
 - Database Connection
-

Phase 2 – Database Setup

Objective

Create database schema.

Tables

uploaded_collections

Store collection metadata.

api_details

Store extracted API information.

generated_scripts

Store generated pytest scripts.

ai_recommendations

Store AI-generated recommendations.

Deliverables

- SQLite Database
 - Database Models
 - Database Relationships
-

Phase 3 – Collection Upload Module

Objective

Upload and validate Postman Collections.

Tasks

- Build Upload Form
- Validate JSON Format
- Validate File Size
- Save Collection Information

Input

Postman Collection JSON

Output

Validated Collection

Deliverables

- Upload Screen
 - Upload API Endpoint
-

Phase 4 – Collection Parser Module

Objective

Extract API information from collections.

Tasks

- Read JSON File
- Extract Requests
- Extract Methods
- Extract URLs
- Extract Headers
- Extract Variables
- Extract Assertions

Output

Structured API Metadata

Deliverables

- Parser Service
 - Parsed Collection Data
-

Phase 5 – Assertion Extraction Engine

Objective

Identify Postman test assertions.

Example Input

Postman Assertion:

```
pm.response.to.have.status(200);
```

Extracted Data

Assertion Type:

Status Code Validation

Expected Value:

200

Deliverables

- Assertion Extractor
 - Assertion Metadata
-

Phase 6 – Gemini AI Conversion Engine

Objective

Convert Postman assertions into pytest assertions.

Workflow

Extracted Assertion



Prompt Construction



Gemini API



Generated Pytest Assertion

Example

Input:

```
pm.response.to.have.status(200);
```

Output:

```
assert response.status_code == 200
```

Deliverables

- Gemini Integration
- Prompt Templates
- Conversion Service

Phase 7 – Pytest Script Generator

Objective

Generate executable pytest scripts.

Tasks

- Create Test Functions
- Create Request Calls
- Create Assertions
- Generate Python Files

Example Output

test_login.py

test_register.py

test_profile.py

Deliverables

- Script Generator
 - Generated Python Files
-

Phase 8 – Validation Engine

Objective

Verify generated output.

Validation Rules

- Syntax Validation
- Missing Assertions Check
- Unsupported Postman Function Detection
- Duplicate Function Detection

Deliverables

- Validation Service
 - Validation Reports
-

Phase 9 – Result Management Module

Objective

Store generated output.

Stored Data

- Uploaded Collections
- Extracted APIs
- Generated Scripts

- AI Recommendations

Deliverables

- Result Storage
 - Result Retrieval APIs
-

Phase 10 – User Interface Development

Objective

Develop application screens.

Screens

- Landing Page
- Upload Page
- Collection Analysis
- Conversion Dashboard
- Code Viewer
- Download Center

Deliverables

- Responsive UI
 - Navigation Flow
-

Phase 11 – Download Module

Objective

Allow users to export generated files.

Export Formats

- Python (.py)
- ZIP Package

Deliverables

- Download Service
 - ZIP Generator
-

Phase 12 – Error Handling

Objective

Handle system failures gracefully.

Error Types

- Invalid JSON
- Parsing Failure
- Gemini API Failure
- Script Generation Failure
- Database Error

Deliverables

- Error Messages
 - Logging System
-

Phase 13 – Testing

Objective

Verify application functionality.

Testing Types

- Unit Testing
- API Testing
- Integration Testing
- End-to-End Testing

Deliverables

- Test Cases
 - Test Results
-

Phase 14 – Final Review

Objective

Prepare project for demonstration.

Checklist

- Upload Working
- Parsing Working
- Gemini Working
- Pytest Generation Working
- Database Working
- Download Working

Deliverables

- Final Application
 - Demo Ready Version
-

System Workflow

User Uploads Collection



Collection Validation



Collection Parsing



Assertion Extraction



Gemini AI Conversion



Pytest Script Generation



Validation



SQLite Storage



Results Display



Download

GitHub Structure

project/

├── app.py

├── requirements.txt

├── database/

├── uploads/

├── generated_scripts/

├── parser/

├── ai_engine/

├── validators/

├── templates/

├── static/

└── README.md

Success Criteria

The project will be considered complete when:

- Postman Collection upload works successfully

- API requests and assertions are extracted correctly
 - Gemini AI converts assertions into pytest code
 - Executable pytest scripts are generated
 - Generated files are stored in SQLite
 - Users can download generated output
 - Complete workflow functions without manual intervention
-

Expected Outcome

The application should successfully convert Postman Collection requests and Postman assertions into executable pytest automation scripts using Gemini AI, significantly reducing manual migration effort for QA engineers and developers.